



Universidade Federal
de Campina Grande

Universidade Federal de Campina Grande
Ciência da Computação

Relatório Final:
Análise de Eficiência de Diferentes Estruturas de Dados
sobre o Sistema ComplementACAO

Disciplina
Estruturas de Dados e Algoritmos

Professor
João Arthur Brunet
joao.arthur@computacao.ufcg.edu.br

Grupo
Alana Vanessa Pimentel Toldo de Andrade — 123210882
Camila Rodrigues de Oliveira Rezende — 123210240
João Victor Muniz Cabral — 123210456
Lorena Nascimento Carvalho — 123211071
Lukas Soares Nascimento — 123210298

Campina Grande – PB
Abril de 2025

1. Introdução.....	3
2. Metodologia.....	3
2.1 Listagem e Avaliação dos Métodos Fundamentais.....	5
2.2 Coleta e Análise dos Dados.....	7
3. Resultados e Análise.....	8
3.1 Análise dos Gráficos.....	8
3.1.1 Adição de uma nova Dica.....	8
3.1.2 Busca de uma Dica que está na primeira posição.....	10
3.1.3 Busca de uma Dica que está no meio da estrutura.....	12
3.1.4 Busca de uma Dica que está no fim da estrutura.....	14
3.1.5 Adição de um novo Usuário.....	16
3.1.6 Busca de um Usuário pelo seu CPF.....	17
3.1.7 Ordenação dos Usuários em Ordem Alfabética.....	19
3.1.8 Ordenação dos Usuários a partir do sua bonificação.....	21
3.2 Comparativo de Desempenho.....	22
3.2.1 Repositórios de Dica.....	22
3.2.2 Repositórios de Usuário.....	23
3.3 Discussão sobre os Resultados.....	23
4. Sugestão de Otimização do Sistema.....	24
5. Conclusão.....	24

1. Introdução

A escolha da estrutura de dados adequada para um sistema pode impactar diretamente seu desempenho e eficiência. Em aplicações que exigem manipulação intensiva de dados, como busca, inserção e ordenação, o tempo de execução dessas operações pode variar significativamente dependendo da estrutura utilizada. O presente trabalho visa analisar o desempenho de diferentes estruturas de dados em um sistema que previamente utilizava HashMap e ArrayList, explorando alternativas como TreeSet e LinkedHashMap para avaliar seus impactos na performance geral do sistema.

Este estudo foi desenvolvido no contexto do Laboratório 5 — ComplementAÇÃO, realizado na disciplina Laboratório de Programação 2. O objetivo deste laboratório foi implementar o ComplementAÇÃO, um sistema voltado para estudantes, com a finalidade de facilitar a gestão de suas atividades complementares no curso. A plataforma permite que os estudantes se cadastrem, registrem suas atividades complementares e possam gerar diferentes tipos de relatórios para melhor visualizar as atividades já realizadas e os créditos acumulados ao concluí-las. Além disso, os estudantes podem registrar dicas sobre as atividades complementares e ganhar pontos por suas ações.

Para isso, foram realizados testes comparativos com diferentes cargas de dados, medindo o tempo de execução de operações fundamentais para a aplicação. Os resultados obtidos foram organizados em gráficos e analisados criticamente para identificar padrões e tendências de comportamento das estruturas de dados utilizadas.

O principal objetivo deste estudo é compreender como a escolha da estrutura de dados influencia a eficiência computacional do sistema, fornecendo subsídios para decisões mais embasadas no desenvolvimento e otimização de aplicações. A partir dessa análise, busca-se determinar quais estruturas apresentam melhor desempenho para cada tipo de operação, considerando cenários com diferentes volumes de dados.

2. Metodologia

Cada parte descrita abaixo pode ser encontrada no seguinte repositório GitHub: [Projeto ComplementACAO](#). Este repositório foi organizado para conter todo o sistema do ComplementACAO, incluindo as novas classes com as diferentes implementações dos repositórios de *Usuario* e *Dica*, os scripts de geração das entradas e as classes que executaram o experimento.

O desenvolvimento deste estudo foi estruturado em etapas bem definidas para garantir uma análise detalhada das diferentes estruturas de dados. Primeiramente, realizou-se uma análise do Laboratório 5 para identificar as estruturas de dados originalmente utilizadas: HashMap, para armazenar objetos do tipo *Usuario*, e ArrayList, para armazenar objetos do tipo *Dica*. A partir disso, foram mapeados os métodos que interagiam com essas estruturas, selecionando os mais relevantes para a análise, especialmente aqueles que envolvem operações de busca, ordenação e inserção. Com essa identificação,

compreendeu-se o papel das estruturas dentro do código e seu impacto na performance da aplicação.

Em seguida, procedeu-se à modificação das estruturas de dados, substituindo as originais por alternativas como TreeMap, HashSet, TreeSet, ArrayList, LinkedHashMap, Deque<ArrayDeque>, HashMap e LinkedList. Para viabilizar essas alterações, ajustaram-se os métodos do sistema para garantir compatibilidade com as novas estruturas. Além disso, houve uma distribuição organizada das responsabilidades entre os membros da equipe para garantir uma implementação eficiente.

Com as modificações concluídas, foi fundamental assegurar o funcionamento adequado do sistema por meio da realização de testes unitários. Para isso, os testes foram atualizados e executados múltiplas vezes a fim de detectar e corrigir possíveis falhas. Posteriormente, definiu-se um conjunto de entradas controladas, geradas a partir de scripts em Python que geraram 500 mil entradas para Usuário e Dica.

A seleção dos métodos a serem analisados foi com base em seu impacto na performance geral do sistema e em como essas operações são comuns em sistemas reais. Métodos como *adicionaUsuario()*, *buscaUsuario()*, *listaEstudantes()* e *listaEstudantesRankingDicas()* na classe Usuário, além de *adicionaDica()* e *buscaDica()* na classe Dica, foram escolhidos para estudo detalhado. Vale salientar que para a análise do método *buscaDica()* foram realizados 3 experimentos: um buscando a primeira dica, ou seja, no início da coleção; um buscando a dica no meio da estrutura, sendo metade da carga; e, uma busca para a última dica.

Na fase de experimentação, cada método foi executado 30 vezes para cada conjunto de entrada, registrando-se os tempos de execução. A partir desses valores, realizou-se a média dos tempos obtidos para cada estrutura e cada cenário analisado. O ambiente de testes foi inicialmente um computador pessoal, no entanto, para garantir uma análise mais robusta e confiável, as execuções finais foram realizadas em uma máquina virtual (VM), possibilitando uma avaliação mais precisa do comportamento das estruturas de dados diante de grandes volumes de entrada.

As configurações da VM utilizada são: 16GB de RAM e 8 vCPUs.

Com esses dados em mãos, foram gerados gráficos comparativos utilizando Python, permitindo visualizar claramente as diferenças de desempenho entre as estruturas. Os gráficos então, foram utilizados para a análise de desempenho de cada estrutura com suas respectivas cargas. Essa análise foi feita por cada integrante do grupo cuja implementação foi sua responsabilidade, conforme a tabela a seguir:

Integrante Responsável	Repositório de <i>Usuário</i> originalmente: <i>HashMap</i>	Repositório de <i>Dica</i> originalmente: <i>ArrayList</i>
Alana	TreeMap	LinkedList
Camila	HashSet	Deque
João Victor	TreeSet	TreeSet
Lorena	ArrayList	HashMap
Lukas	LinkedHashMap	LinkedHashMap

2.1 Listagem e Avaliação dos Métodos Fundamentais

Nesta etapa, foram listados e avaliados os principais métodos do sistema que interagem diretamente com as estruturas de dados centrais da aplicação: *HashMap* e *ArrayList*. A análise concentrou-se em operações fundamentais como inserção, busca e ordenação, por serem as mais impactadas pela escolha da estrutura de dados.

No repositório de usuários, implementado com *HashMap*, foram selecionados quatro métodos para análise: *adicionaUsuario()*, responsável por adicionar um novo usuário ao sistema, garantindo a unicidade com base no CPF; *buscaUsuario()*, que realiza a busca por um usuário cadastrado a partir de seu CPF; *listaEstudantes()*, que retorna os usuários em ordem alfabética de nome; e *listaEstudantesRankingDicas()*, que ordena os estudantes com base na pontuação de bonificação por dicas, aplicando um critério secundário de ordenação alfabética em caso de empate.

No repositório de dicas implementado com *ArrayList*, os métodos analisados foram: *adicionaDica()*, que insere uma nova dica no final da lista mantendo a ordem de inserção; e *buscaDica()*, que retorna uma dica presente em uma posição específica da lista. A escolha desses métodos se deu por sua relevância no uso cotidiano do sistema e por representarem padrões comuns de manipulação de dados em aplicações reais.

Considerando a implementação original e suas estruturas, segue abaixo a análise de custo de cada método escolhido.

1. Implementação original de *UsuarioRepository* em *HashMap*

a) Dentro do método *adicionaUsuario(Usuario estudante)*:

- i) Uma validação do objeto passado por parâmetro, para garantir que este não é nulo. Isso tem custo constante de $O(1)$.
- ii) A verificação se o *HashMap* já contém a chave CPF do estudante, pois na descrição do sistema há a restrição de que não deve ser possível adicionar dois usuários com mesmo CPF. Como estudantes é um *HashMap*, a operação *containsKey* tem complexidade média de $O(1)$.
- iii) Inserção propriamente dita do usuário com método *put()*, que também possui complexidade $O(1)$.
- iv) O custo no pior caso é $O(1)$.

b) Dentro do método *listaEstudantes()*:

- i) Verificação se há algum estudante já cadastrado. Se o HashMap estiver vazio, retorna um array do tipo String, vazio. Complexidade de $O(1)$, pois apenas verifica e retorna.
- ii) Ordenação alfabética dos usuários a partir da criação de um TreeSet com os valores do HashMap estudantes. O TreeSet ordena os elementos automaticamente de acordo com a ordem natural dos objetos do tipo Usuário, que neste caso é alfabética. A complexidade desta operação é $O(n \log n)$.
- iii) Por fim, há a conversão do TreeSet ordenado em um array de strings. A conversão percorre todos os elementos do TreeSet e chama o método *toString()* de cada usuário. Logo sua complexidade é $O(n)$.
- iv) Por fim, o custo no pior caso é $O(n \log n)$.

c) Dentro do método *buscaUsuario(Usuario estudante)*:

- i) Busca pelo usuário com o cpf que é a chave no HashMap, através do método *get(cpf)*. Essa busca possui complexidade $O(1)$, pois é uma busca direta através da chave informada.
- ii) Validação do objeto retornado pelo *get(cpf)*, pois num HashMap, caso a chave não esteja vinculada a nenhum valor, o valor retornado é nulo. Por isso, essa validação verifica se o objeto é nulo ou não e se a senha cadastrada é a do respectivo usuário, para então retornar o objeto buscado. Essa validação também custa $O(1)$.
- iii) Portanto o custo total deste método é $O(1)$.

d) Dentro do método *listaEstudantesRankingDicas()*:

- i) Verificação se há algum estudante já cadastrado. Se o HashMap estiver vazio, retorna um array do tipo String, vazio. Complexidade de $O(1)$, pois apenas verifica e retorna.
- ii) De forma similar ao *listaEstudantes()*, este método também cria um TreeSet, mas ordenado de forma reversa a partir de um atributo específico de *Usuario*: o seu valor de bonificação. Essa bonificação é dada pela pontuação que o usuário possui a partir de Dicas adicionadas no sistema. A complexidade desta operação é $O(n \log n)$, sendo n a quantidade de estudantes. Por fim, há a conversão do TreeSet ordenado em um array de strings. A conversão percorre todos os elementos do TreeSet e chama o método *toString()* de cada usuário. Logo sua complexidade é $O(n)$.
- iii) Por fim, o custo no pior caso é $O(n \log n)$.

2. Implementação original de *DicaRepository* em *ArrayList*

a) Dentro do método *buscaDica(int posicao)*:

- i) Validação se a posição passada por parâmetro está dentro dos limites do array, esta validação envolve uma comparação simples, que é uma operação de tempo constante $O(1)$.
- ii) Acesso ao elemento na posição pelo método *get()*, por ser uma busca direta pelo índice, também tem custo constante $O(1)$.
- iii) Sendo assim, o custo total do método de busca é $O(1)$.

b) Dentro do método *adicionaDica(Dica dica)*:

- i) Validação do objeto *Dica* passado por parâmetro, é uma operação simples de custo constante, $O(1)$.
- ii) A operação de adição de fato em um *ArrayList* é de tempo constante amortizado. Em casos normais, adicionar um elemento no final da lista é $O(1)$. No entanto, ocasionalmente, a lista pode precisar ser redimensionada, o que envolve copiar os elementos para um novo array maior. Esse redimensionamento ocorre em $O(n)$, mas como é raro, a complexidade média (amortizada) da operação de adição permanece $O(1)$.
- iii) O custo total do método de adição é $O(1)$ amortizado.

2.2 Coleta e Análise dos Dados

Para a realização do experimento analítico, foi implementado e utilizado um script em Python responsável pela geração de dados. Esse script produziu, de forma aleatória, strings que serviram como valores para os atributos nome e CPF, possibilitando a criação dos objetos *Usuario* necessários para a análise. Os dados gerados foram suficientes para atender às diferentes cargas de teste. Além disso, foi garantida uma taxa de repetição de 20% nos nomes dos usuários, enriquecendo o universo amostral e beneficiando a avaliação do método *listaUsuarios()*, cuja função é listar os usuários do sistema em ordem alfabética.

Adicionalmente, no contexto do sistema de *Dica*, onde cada *Usuario* acumula pontuação pelas dicas adicionadas, foi gerado um número aleatório — utilizando a classe *java.util.Random* — e atribuído ao atributo pontuação de cada objeto *Usuario*. Isso proporcionou maior diversidade à base de dados de objetos *Dica* e *Usuario*, aproximando a amostra de um cenário mais realista e contribuindo diretamente para a análise do método *listaUsuariosPorRanking()*.

Durante a execução dos métodos de adição, busca, listagem e listagem por ranking de *Usuario* e *Dica* nas estruturas de dados implementadas, foi aferido o tempo em nanossegundos entre o início e o fim da execução de cada método, em cada estrutura. Ao final de cada execução, foi calculada a média de tempo com base em 30 repetições, e os resultados foram registrados em arquivos de texto (.txt), os quais serviram de base para a geração dos gráficos do experimento.

Por fim, a implementação e execução deste experimento demandaram elevado nível de atenção e dedicação por parte dos envolvidos. Isso se deve, em

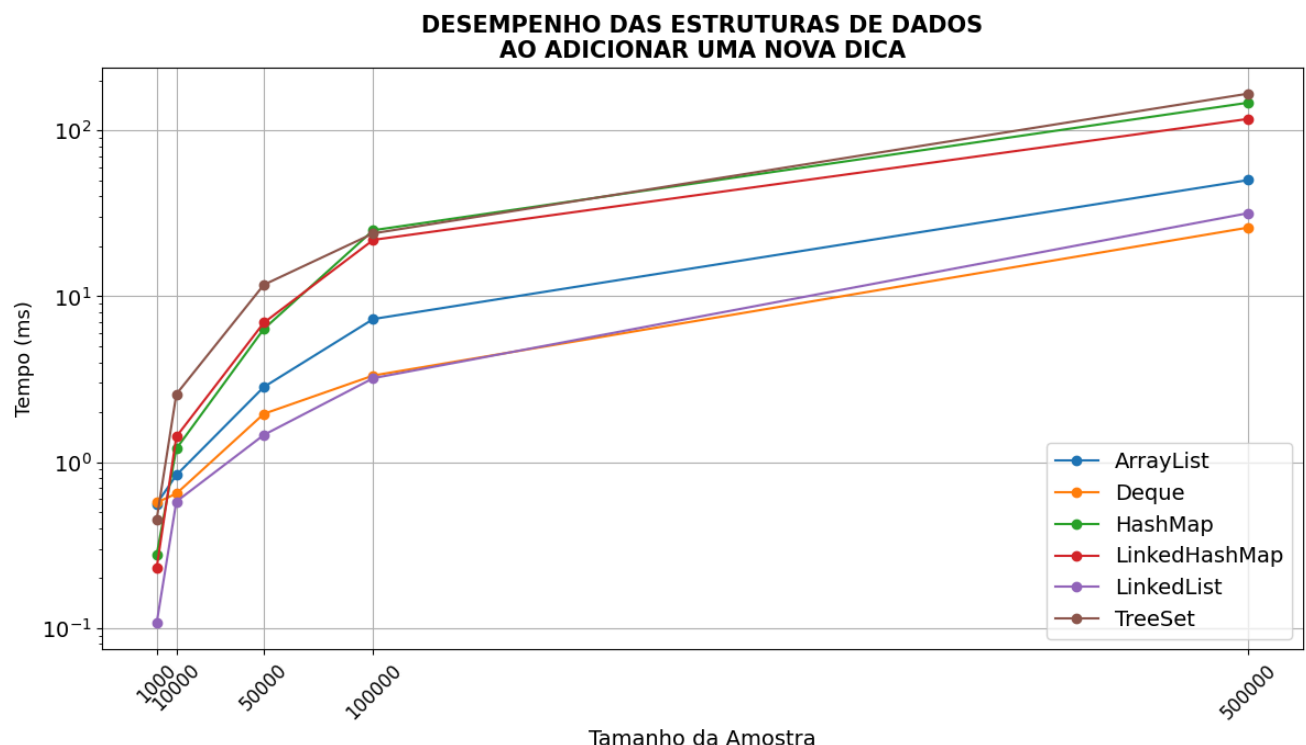
grande parte, à necessidade de pesquisas além do escopo acadêmico, visando ao aprendizado antecipado de recursos relacionados à programação concorrente, à geração automatizada de dados, à manipulação de estruturas de dados e ao uso correto do terminal (shell). A complexidade do projeto experimental-analítico foi acentuada por essas exigências. Ademais, a preocupação com a adequação do ambiente de execução foi essencial para o cumprimento dos prazos estabelecidos, já que, devido à elevada carga de dados processados, foi necessário dispor de um ambiente dedicado para garantir o desempenho e a confiabilidade das análises realizadas.

3. Resultados e Análise

3.1 Análise dos Gráficos

Nesta seção, serão apresentados e discutidos os resultados obtidos pela execução dos experimentos nas operações fundamentais. Cada gráfico reflete o tempo médio de execução dessas operações em função do aumento da carga de dados, permitindo uma comparação clara entre os desempenhos das estruturas. A análise detalha o comportamento individual de cada estrutura, identificando padrões, possíveis gargalos e justificando o desempenho observado com base nas características internas de implementação de cada uma. A seguir, discutimos os resultados obtidos para cada estrutura, com foco especial nas variações de desempenho conforme a carga de entrada.

3.1.1 Adição de uma nova Dica



HashMap

- O uso da capacidade inicial padrão (de 16) causou múltiplos rehashings nas primeiras cargas (1.000 a 10.000 elementos), elevando o tempo de execução.
- A partir de 50.000 elementos, o desempenho estabilizou, com inserções próximas de custo constante amortizado.
- O resultado reforça a importância de definir uma capacidade inicial adequada ao lidar com grandes volumes de dados.

Deque <ArrayDeque>

- A adição de elementos no final apresenta complexidade constante $O(1)$, devido ao suporte eficiente para inserções nas extremidades.
- A curva do gráfico se mantém abaixo das outras estruturas, mostrando desempenho superior à medida que o tamanho da amostra aumenta.
- O crescimento suave e previsível nos tempos de execução, mesmo com grandes amostras (500.000 elementos), destaca a eficiência do Deque para inserções.
- O Deque é uma excelente escolha para cenários com inserções frequentes nas extremidades.

LinkedHashMap

- A inserção apresenta complexidade média $O(1)$, com custo adicional para manutenção da ordem de inserção, resultando em tempo de execução estável e previsível.
- A curva de desempenho cresce de forma aproximadamente linear na escala log-linear, alinhada com a complexidade $O(1)$.
- Até 50.000 elementos, o desempenho foi levemente inferior ao do HashMap, mas superou-o após esse ponto, mostrando ser assintoticamente melhor em grandes cargas.
- Comparado a outras estruturas (como ArrayList, LinkedList, e Deque<ArrayDeque>), o desempenho foi inferior, especialmente em amostras grandes.
- O LinkedHashMap é eficiente para inserções em larga escala, preservando a ordem de inserção, mas não é a melhor opção para grandes volumes, em relação a outras estruturas mais rápidas.

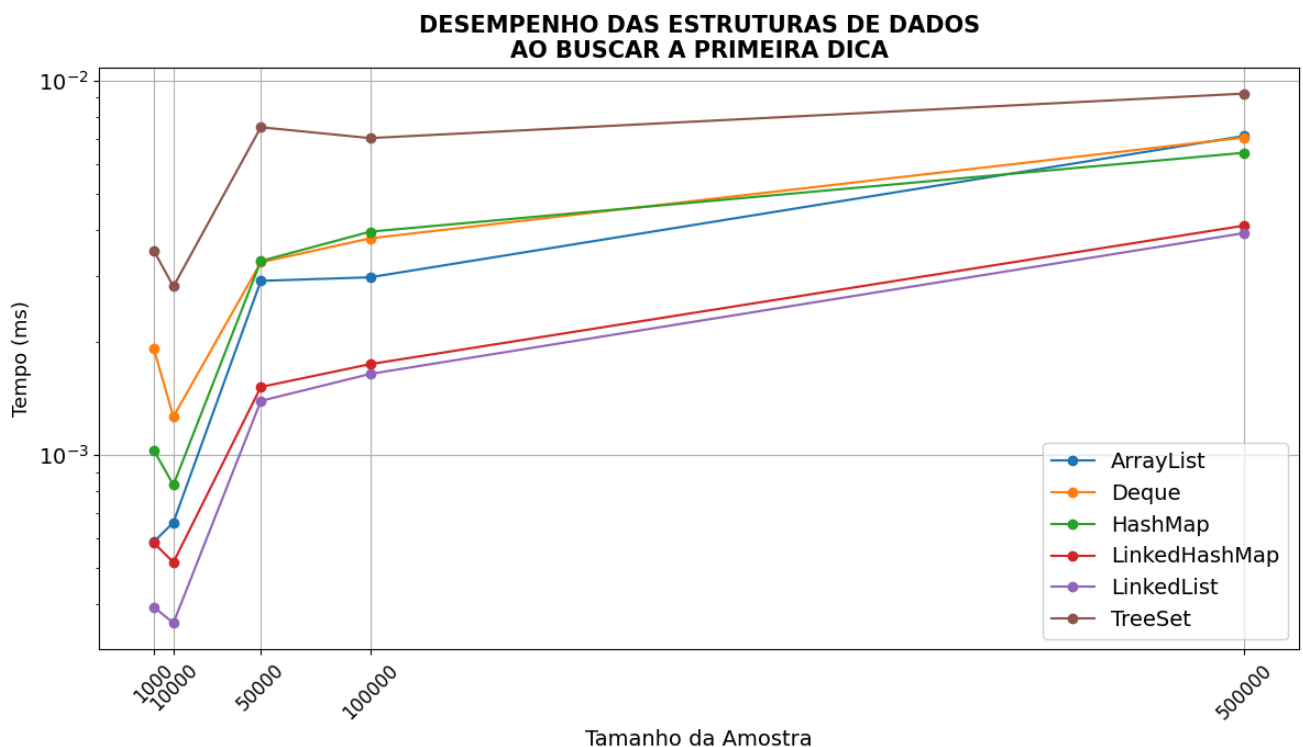
LinkedList

- Razões entre as amostras: ao multiplicar a entrada por 5 (10k para 50k), o tempo aumentou 2,5 vezes. Quando a entrada dobrou (50k para 100k), o aumento foi de 2,2 vezes. Por fim, ao multiplicar por 5 novamente (100k para 500k), o tempo saltou 9,9 vezes.
- O desvio de crescimento para a maior carga pode ter sido causado pela alocação de memória, gerenciamento interno da JVM.
- O gráfico reforça um comportamento linear, sendo importante ressaltar que a estrutura apresentou o segundo melhor desempenho nesse método.

TreeSet

- A inserção de um elemento ocorre em uma árvore binária de busca balanceada (geralmente árvore rubro-negra), com complexidade $O(\log n)$.
- A altura da árvore é logarítmica em relação ao número de elementos, resultando em tempo de execução logarítmico para a inserção.
- A operação de obter o tamanho é $O(1)$, devido ao contador interno de elementos.
- A complexidade total da operação de inserção pode ser expressa como $O(\log n)$.

3.1.2 Busca de uma Dica que está na primeira posição



Deque<ArrayDeque>

- A busca é realizada por laço *for*, percorrendo a estrutura elemento por elemento, o que resulta em complexidade linear $O(n)$.
- Inicialmente, entre 1.000 e 10.000 elementos, houve pequenas oscilações nos tempos de execução, possivelmente devido a alocação dinâmica ou interferências externas.
- A partir de 50.000 elementos, o desempenho se estabilizou, com crescimento linear conforme o aumento da amostra.
- O comportamento reflete a complexidade $O(n)$ esperada para a operação de busca no Deque.

HashMap

- Inicialmente, houve oscilações no tempo de busca, possivelmente devido a colisões, má distribuição de hashes ou interferência da Garbage Collector.

- Após 50.000 elementos, o desempenho estabilizou, com buscas rápidas e consistentes.
- A busca por uma chave fixa (chave = 1) favorece a eficiência da função de hash, resultando em comportamento compatível com a complexidade média $O(1)$.

LinkedHashMap

- A busca apresentou tempos de resposta baixos e consistentes, mantendo complexidade $O(1)$ devido à estrutura baseada em tabela de hash, que preserva a ordem de inserção.
- Para cargas menores que 50.000, a complexidade foi linear $O(n)$, mas a partir desse ponto, o desempenho estabilizou em $O(1)$, refletindo uma busca rápida e eficiente.
- Destaca-se como uma escolha ideal para equilíbrio entre rapidez de acesso e preservação da ordem dos elementos.

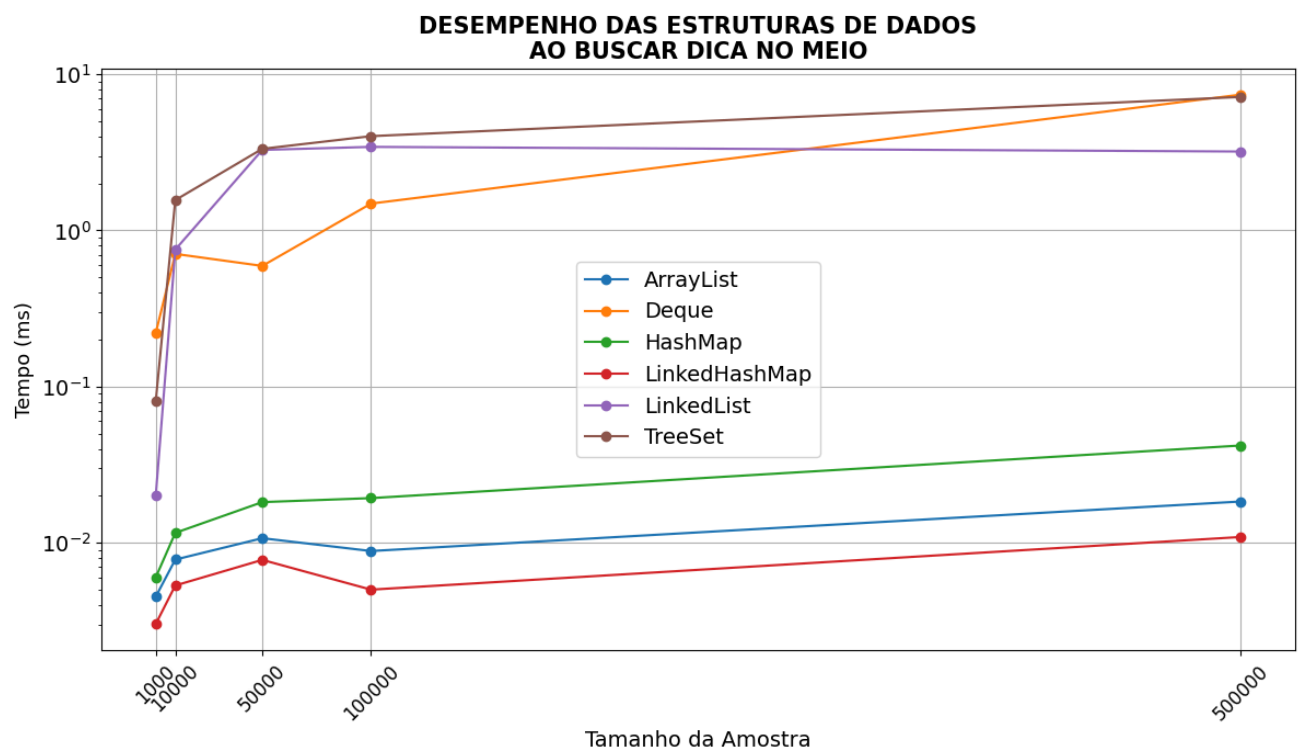
LinkedList

- Razões entre as amostras: ao multiplicar a entrada por 5, o tempo aumentou 3,9 vezes; com a duplicação da entrada, o crescimento foi de 1,2 vezes; e ao multiplicar novamente por 5, o tempo cresceu 2,3 vezes.
- O gráfico demonstra que o acesso à primeira posição da LinkedList é praticamente constante, com pequenas variações causadas por fatores externos.
- É confirmado o comportamento $O(1)$ para buscar a primeira dica, sendo a LinkedList a estrutura com melhor desempenho nesse método.

TreeSet

- Sabendo que o TreeSet insere seus elementos de forma ordenada, buscar o primeiro elemento que foi inserido torna-se uma tarefa ineficaz para a estrutura. Uma vez que, não há uma comparação definida para que a Dica seja ordenada adequadamente, a primeira que foi inserida nem sempre ficará na primeira posição. Por causa disso, a função apresenta inconstância no intervalo [10000, 100000] e uma certa constância no intervalo]100000, 500000] devido ao processo de busca com complexidade $O(\log n)$.
- A Busca no TreeSet é realizada com base na altura da árvore, a mesma realiza operações de rotações para manter-se balanceada e com menor valor possível de altura.

3.1.3 Busca de uma Dica que está no meio da estrutura



Deque<ArrayDeque>

- A busca apresenta comportamento linear $O(n)$, com tempos de execução aumentando proporcionalmente ao tamanho da amostra.
- O Deque não oferece acesso posicional eficiente, sendo implementado como um array circular que requer a busca sequencial até o elemento desejado.
- Para amostras pequenas (1.000 a 10.000), o desempenho foi razoável (abaixo de 1 ms), mas a partir de 50.000 elementos, o tempo ultrapassou 1 ms, chegando a quase 10 ms para 500.000 elementos.
- Esse comportamento reflete a complexidade $O(n)$ de busca em estruturas não indexadas.
- O Deque é eficiente para operações nas extremidades, mas não é ideal para buscas por posição, já que não oferece acesso direto.

HashMap

- Apresentou tempos de busca baixos e crescimento suave, mesmo com grandes volumes de dados.
- O tempo se manteve abaixo de 0,05 ms, sem picos ou variações significativas.
- Resultados indicam poucas ou nenhuma colisão, alinhando-se à complexidade média $O(1)$.
- Reforça a eficiência do HashMap em buscas pontuais, mesmo em cenários com alta carga.

LinkedHashMap

- O desempenho foi excepcionalmente estável e eficiente, com tempos de execução baixos e praticamente constantes, refletindo a complexidade média $O(1)$ de acesso por chave.
- Mesmo com a sobrecarga de preservação da ordem de inserção, o impacto no tempo de busca foi insignificante.
- Superou significativamente outras estruturas como LinkedList, TreeSet e Deque, que têm buscas lineares ou logarítmicas.
- Reforça o LinkedHashMap como uma excelente escolha para operações frequentes de acesso direto a dados, mesmo em grandes volumes.

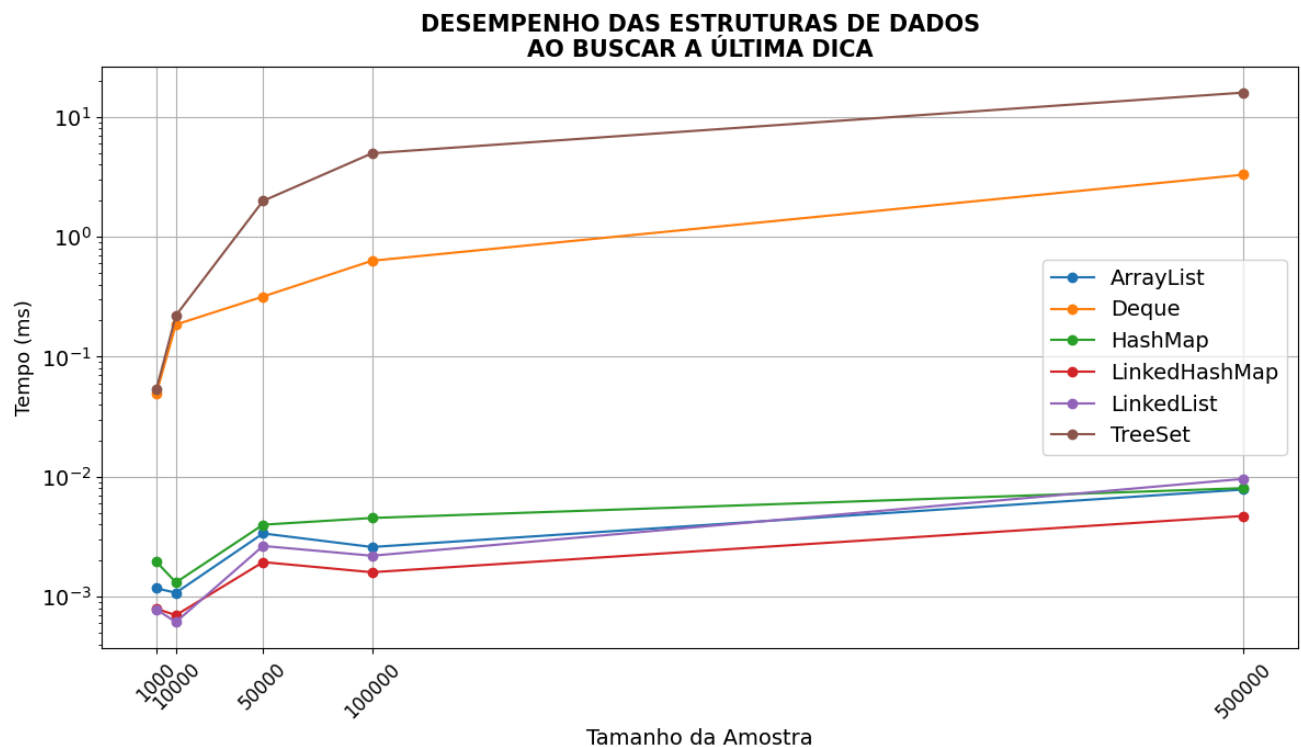
LinkedList

- Razões entre as amostras: ao multiplicar a entrada por 5, o tempo médio aumentou cerca de 4,3 vezes; ao dobrar a entrada, o tempo praticamente se manteve, com uma razão de 1,1; e ao multiplicar novamente por 5 a entrada, houve uma leve queda no tempo; com razão de 0,9.
- A partir do gráfico, é observado que o acesso ao meio da LinkedList é linear — $O(n)$ — e ineficiente, sendo a terceira pior estrutura nesse método.

TreeSet

- A validação é realizada em tempo constante $O(1)$.
- O método add tem complexidade $O(\log n)$, devido à estrutura de árvore binária de busca balanceada.
- Para buscar a dica no meio, é necessário iterar sobre os elementos, o que leva $O(n)$ no pior caso, já que seria preciso percorrer até $n/2$ elementos.
- A operação size() é $O(1)$, pois o tamanho é mantido internamente.
- A complexidade total da busca da dica do meio em um TreeSet é $O(n)$.

3.1.4 Busca de uma Dica que está no fim da estrutura



Deque<ArrayDeque>

- O desempenho mostra uma degradação gradual, com tempos de execução aumentando proporcionalmente ao tamanho da amostra, refletindo uma complexidade linear $O(n)$.
- Como o acesso é aleatório, a busca precisa percorrer os elementos sequencialmente até encontrar a posição desejada.
- Nos primeiros pontos do gráfico (1.000 a 10.000 elementos), o impacto do custo sequencial é pequeno, com tempos de busca abaixo de 1 ms.
- À medida que a amostra cresce para 50.000, 100.000 e 500.000 elementos, o tempo de execução aumenta consideravelmente, superando 10 ms no maior volume testado.
- O comportamento reflete a complexidade típica de estruturas lineares não indexadas em cenários de acesso aleatório.

HashMap

- O desempenho se manteve estável e eficiente, com tempos de resposta baixos e consistentes, variando de 0,0026 ms (para carga de 1.000) até 0,0084 ms (para carga de 500.000).
- A progressão foi quase linear no eixo logarítmico, sem gargalos visíveis, mesmo com o aumento no volume de dados.
- A chave, mesmo sendo a maior possível, não afeta negativamente a dispersão, garantindo tempos de acesso semelhantes para qualquer chave, sem penalização pela posição.

- O acesso ao HashMap é feito via cálculo de índice, sem necessidade de percorrer a estrutura sequencialmente, resultando em tempos de busca constantes.

LinkedHashMap

- O desempenho foi excelente, com tempos de execução extremamente baixos em todas as escalas de amostra testadas.
- A estrutura se beneficiou do acesso direto por chave, mantendo complexidade média $O(1)$ mesmo ao buscar o elemento mais recentemente adicionado.
- A preservação da ordem de inserção não impactou negativamente o tempo de busca.
- Superou outras estruturas como Deque e TreeSet, que apresentaram crescimento acentuado no tempo de execução.
- Ideal para aplicações que exigem tanto a preservação da ordem de inserção quanto alto desempenho na recuperação de dados.

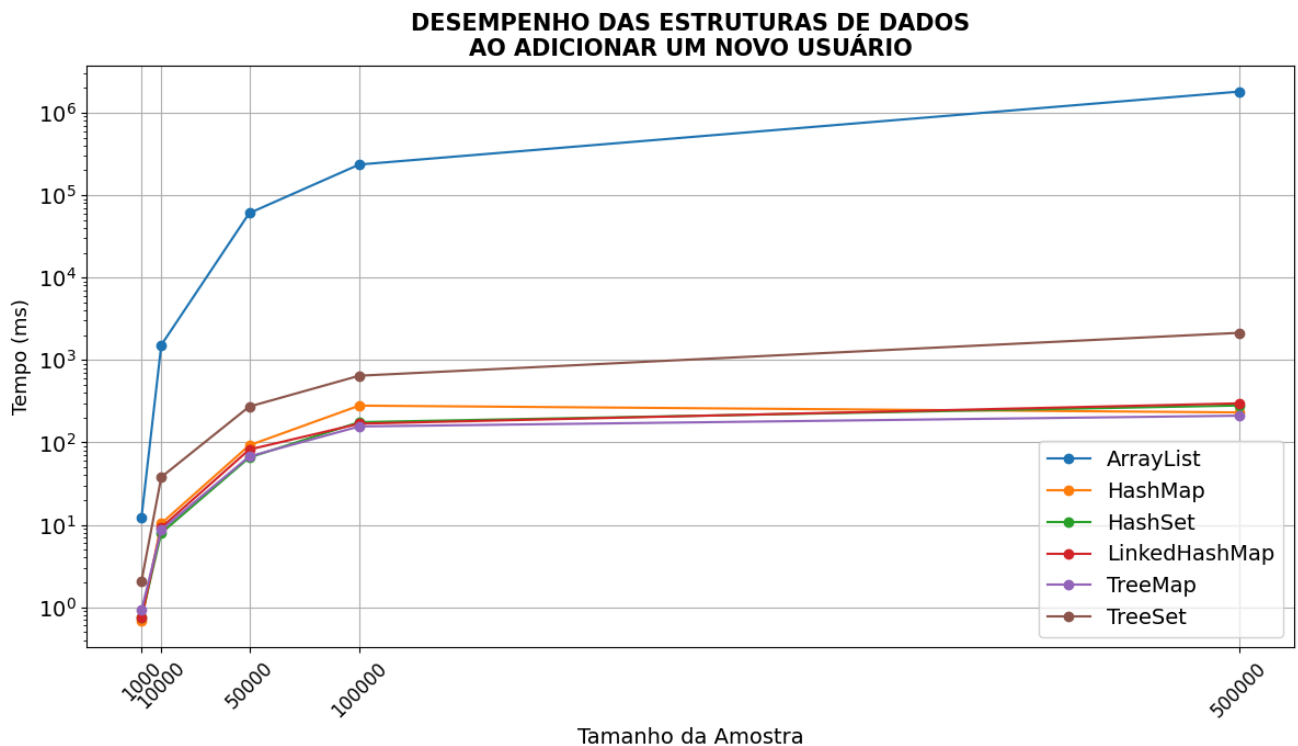
LinkedList

- Razões entre as amostras: ao multiplicar a entrada por 5, o tempo subiu 4,3 vezes; dobrando a entrada, houve uma redução no tempo médio, de 1,2 vezes; ao multiplicar novamente por 5, o tempo cresceu em 4,3 vezes.
- O gráfico demonstra um comportamento linear, porém os valores mostram que houveram oscilações nos tempos de execução. Entre todas as estruturas, o LinkedList ficou em quarta posição para esse método.

TreeSet

- No intervalo [1000, 100000] de tamanho de amostra há uma elevação considerável do tempo de execução da busca. A razão desta curva de crescimento é a organização interna da estrutura (balanceamento).
- Quanto maior a carga, maior o tempo de busca, pois ela é executada com base na altura da árvore. Sendo a função do tempo de execução da busca na estrutura $O(\log n)$.

3.1.5 Adição de um novo Usuário



ArrayList

- No ArrayList, a verificação com *contains()* impõe uma complexidade $O(n)$, já que percorre toda a lista para garantir unicidade.
- A inserção ao final da lista é $O(1)$, mas seu impacto é desprezível diante do custo da verificação.
- O gráfico mostra crescimento acentuado no tempo de execução: ~11 ms para 1.000 elementos e 1.500.000 ms (cerca de 25 minutos) para 500.000 elementos
- Torna-se impraticável para grandes volumes de dados quando há necessidade de garantir unicidade.
- Reforça a inadequação do ArrayList para operações com validação de duplicatas em larga escala.

HashSet

- Tanto *contains()* quanto *add()* possuem complexidade média $O(1)$.
- A verificação de duplicidade é eficiente graças ao uso da tabela de hash e implementação adequada de *hashCode()* e *equals()* (com base no CPF).
- O gráfico mostra crescimento suave no tempo de execução: ~9 ms com 1.000 elementos, e menor 300 ms com 500.000 elementos
- Dessa forma, o HashSet é ideal para adicionar elementos únicos rapidamente, mesmo em grandes volumes de dados.

LinkedHashMap

- O LinkedHashMap apresenta desempenho estável e eficiente, mesmo com aumento do volume de dados.
- A complexidade média da operação de inserção é $O(1)$.

- Leve custo adicional pela preservação da ordem de inserção.
- Desempenho ligeiramente inferior ao HashMap, mas muito superior ao ArrayList.
- Mais eficiente que estruturas como TreeSet e ArrayList para inserções.
- Ideal para inserções rápidas com ordem preservada, mesmo em grandes amostras

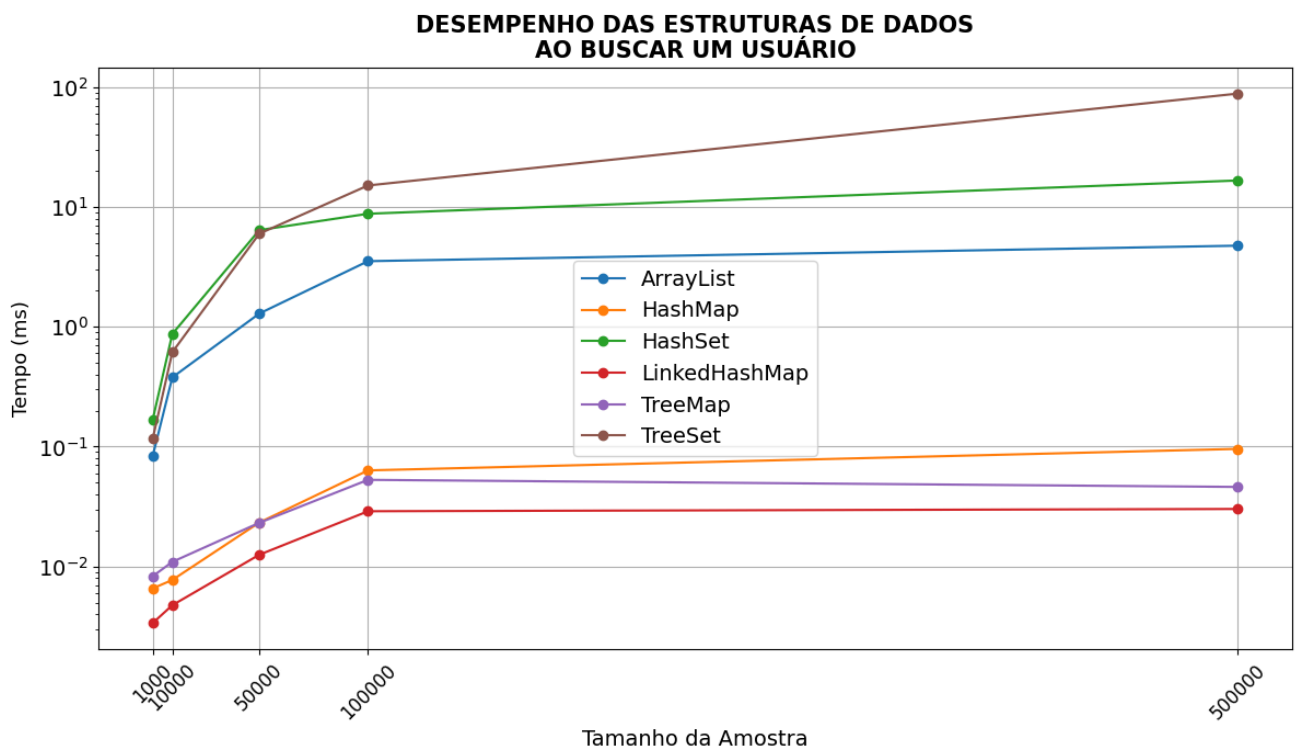
TreeMap

- Razões entre as amostras: ao multiplicar a entrada por 5, o tempo subiu 7,8 vezes; ao dobrar a entrada, houve um aumento de 2,3 vezes no tempo médio; e ao multiplicar por 5 novamente, o tempo subiu apenas 1,3 vezes.
- O gráfico denota um crescimento quase linear com fator logarítmico, o que é esperado de estruturas balanceadas como o TreeMap. Foi a estrutura com melhor desempenho para a adição de usuários.

TreeSet

- É notório que a função do tempo de execução é $O(\log n)$, isso se dá porque o TreeSet é implementado com base em uma árvore balanceada.
- A função aumenta de forma mais evidente no intervalo [1000, 100000], após isso a curva de crescimento diminui pois, ao passo que durante todo o processo de adição de novos usuários a árvore realiza operações para manter-se balanceada.

3.1.6 Busca de um Usuário pelo seu CPF



ArrayList

- Apresenta crescimento linear no tempo de resposta conforme o número de elementos aumenta (complexidade $O(n)$).
- Podemos ver o crescimento da curva: 1.000 elementos → 0,08 ms; 500.000 elementos → 4,93 ms.
- Bom desempenho para coleções pequenas, mas ineficiente para grandes volumes.
- Pouco indicado para sistemas com buscas frequentes e exigência de performance

HashSet

- O HashSet utiliza hashing para inserção e busca eficiente, mas a busca por CPF foi feita manualmente via o método `contains()`, o que força a iteração sobre todos os elementos.
- Embora o HashSet tenha complexidade constante $O(1)$ para buscas diretas, a busca por CPF resulta em uma complexidade linear $O(n)$, pois é necessário verificar cada elemento individualmente.
- O gráfico mostra um crescimento proporcional ao tamanho da amostra, com o tempo de busca aumentando conforme o número de elementos cresce, indicando a complexidade $O(n)$.

LinkedHashMap

- O LinkedHashMap demonstrou excelente desempenho, com tempos consistentemente baixos mesmo em grandes volumes de dados.
- O LinkedHashMap demonstrou ser a melhor estrutura para esse contexto com tempos de execução significativamente menores do que os observados em estruturas como HashSet, TreeSet e ArrayList. Esse resultado confirma a eficiência do LinkedHashMap em operações de busca, aliando rapidez ($O(1)$) ao benefício da preservação da ordem de inserção, característica que pode ser essencial em diversas aplicações.

TreeMap

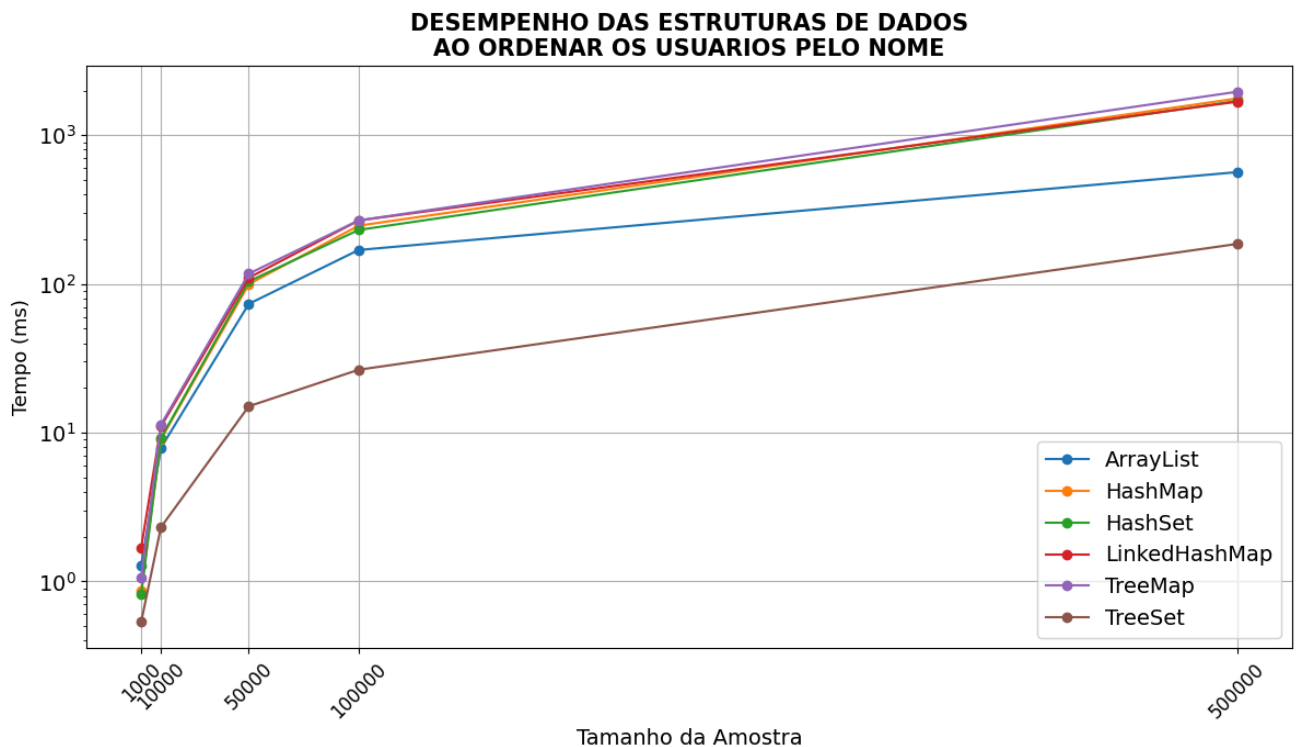
- Razões entre as amostras: ao multiplicarmos a entrada por 5, houve um crescimento do tempo de 2,1 vezes; ao dobrarmos a entrada, o crescimento do tempo praticamente se manteve, com razão de 2,3; porém ao multiplicar por 5 novamente a entrada, o tempo apresentou uma leve redução, com razão de 0,9.
- O gráfico e os valores indicam um crescimento sublinear no tempo de busca por CPF, o que é coerente com o esperado para a estrutura.
- Além disso, o TreeMap obteve o segundo melhor desempenho nessa operação.

TreeSet

- A busca na estrutura é realizada em $O(\log n)$, sendo n o tamanho da altura da árvore.
- Execuções internas são realizadas na estrutura para mantê-la com a

altura mínima possível, por isso, o início da função no intervalo [1000, 100000] apresenta um crescimento maior do que no intervalo]100000, 500000]. Pois a estrutura interna expande sua capacidade de armazenamento para os lados da árvore, mantendo-a com uma altura que varia pouco.

3.1.7 Ordenação dos Usuários em Ordem Alfabética



ArrayList

- O ArrayList utiliza o algoritmo de ordenação `Arrays.sort()`, que geralmente implementa o TimSort, um algoritmo híbrido derivado do merge sort e insertion sort.
- A complexidade do algoritmo de ordenação utilizado é $O(n \log n)$, com um custo adicional $O(n)$ para a conversão da lista em um array ao final do processo. A complexidade dominante é $O(n \log n)$ devido à ordenação, enquanto o custo adicional da conversão é superado.
- A curva mostra um crescimento estável e progressivo do tempo de ordenação à medida que a carga aumenta, com o tempo de execução se aproximando de 520 ms para a maior carga testada. Esse comportamento é consistente com a complexidade $O(n \log n)$ do algoritmo de ordenação utilizado.

HashSet

- O HashSet, sendo eficiente para operações de inserção e busca ($O(1)$ em média), não é uma estrutura projetada para ordenação. No entanto, no contexto da análise, a ordenação dos elementos é realizada por meio da criação de um TreeSet, o que resulta na reorganização completa dos elementos conforme a ordem natural (alfabética, no caso de strings).

- A operação de ordenação tem complexidade $O(n \log n)$, uma vez que cada elemento precisa ser reposicionado corretamente na árvore binária de busca (geralmente um `TreeSet`), que tem essa complexidade para inserção e ordenação.
- O gráfico mostra um aumento significativo no tempo de execução à medida que o volume de dados cresce. Com 500.000 elementos, o tempo de ordenação supera 1 segundo, indicando que a operação de ordenação, em particular, se torna um gargalo de desempenho à medida que a carga aumenta.

LinkedHashMap

- O `LinkedHashMap` mantém a ordem de inserção, mas para ordenar os elementos por nome, é necessário extrair os itens para uma lista e aplicar a ordenação, resultando em complexidade $O(n \log n)$.
- A ordenação tem custo $O(n \log n)$, devido ao algoritmo de ordenação aplicado após a extração dos dados.
- O tempo de execução cresce significativamente conforme a amostra aumenta, superando 1000 ms para 500.000 registros.
- Embora eficiente para inserção e busca ($O(1)$), o `LinkedHashMap` não é ideal para ordenação de grandes volumes de dados, sendo superado por estruturas como o `TreeSet`, que já mantém dados ordenados de forma mais eficiente.

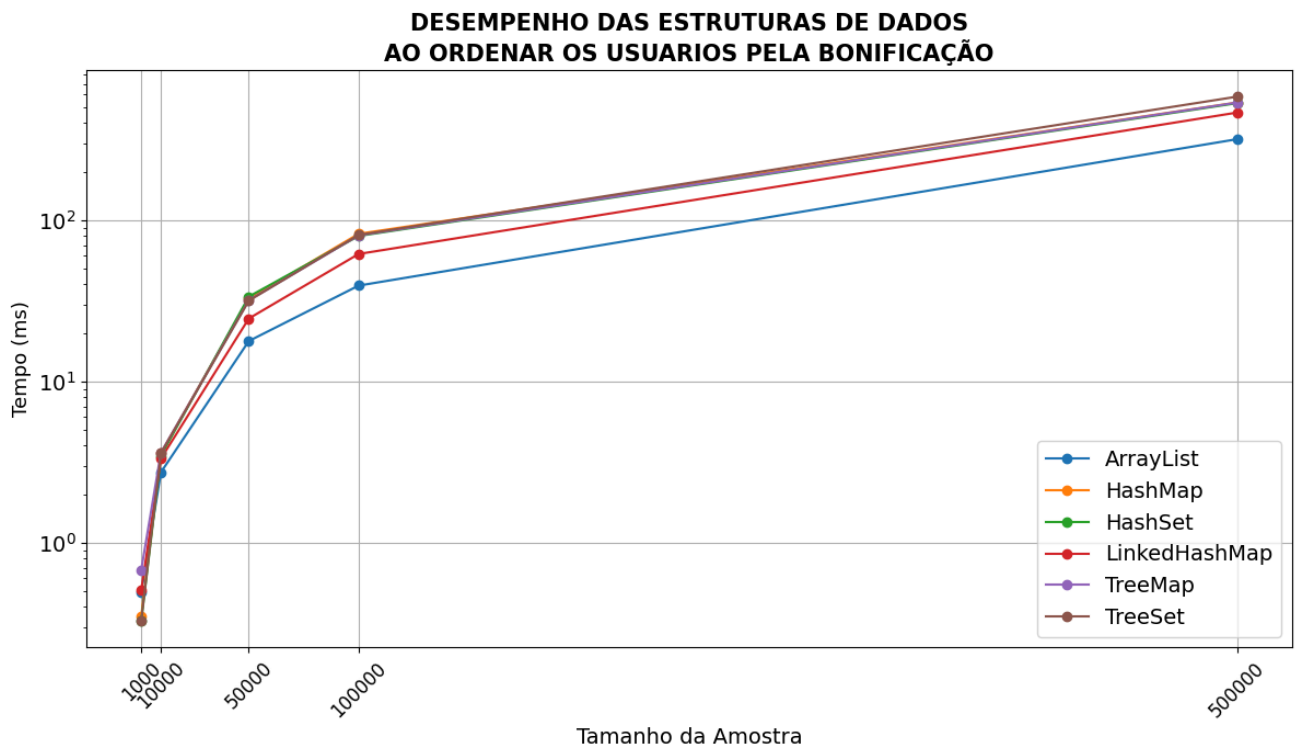
TreeMap

- Razões entre as amostras: multiplicando a entrada por 5, o tempo cresceu em uma razão de 10,3; depois, ao dobrar a entrada, a razão entre os tempos médios foi de 2,3; e ao multiplicar por 5 novamente, o tempo subiu 7,3 vezes.
- O `TreeMap` mantém os usuários ordenados por CPF (chave de busca), não pelo nome, então é necessária a aplicação de um algoritmo de ordenação baseado em comparação, com complexidade $O(n \log n)$, o que explica o crescimento dos tempos de execução.
- Nesse método, o `TreeMap` apresentou o pior desempenho entre as estruturas analisadas.

TreeSet

- O `TreeSet` insere os armazena os seus elementos de forma ordenada em tempo de complexidade $O(\log n)$, por isso apresenta melhor desempenho em relação às demais estruturas levando apenas 185,968 ms para execução com a maior carga analisada.
- A função de desempenho é uma curva logarítmica em função do tamanho da carga armazenada.

3.1.8 Ordenação dos Usuários a partir do sua bonificação



ArrayList

- O ArrayList utiliza o `Arrays.sort()` com TimSort e comparador reverso para ordenar os usuários pela bonificação, mantendo a eficiência.
- A ordenação segue a complexidade $O(n \log n)$, com desempenho eficiente e estável conforme o aumento da carga.
- Avaliando o comportamento no gráfico vemos que os tempos variaram de 2,8 ms para 1.000 elementos a 277 ms para 500.000 elementos, refletindo a complexidade esperada.

HashSet

- Como o HashSet não mantém a ordenação, os dados são transferidos para um TreeSet com comparador baseado na bonificação e nome.
- A construção do TreeSet impõe complexidade $O(n \log n)$, devido à ordenação durante as inserções.
- O tempo de execução cresce de < 5 ms (1.000 elementos) para > 100 ms (500.000 elementos), demonstrando aumento estável.
- Embora eficiente em inserções e buscas, o HashSet exige uma etapa extra de reordenação para tarefas como esta, o que limita seu desempenho em cenários que demandam ordenação.

LinkedHashMap

- Como o LinkedHashMap mantém apenas a ordem de inserção, é necessário extrair os elementos e ordená-los manualmente, como na ordenação por nome. Essa ordenação exige um algoritmo externo, resultando em complexidade $O(n \log n)$.

- Pela curva do gráfico podemos ver que o tempo de execução cresce com o volume de dados, chegando a quase 1000 ms com 500.000 usuários.
- Então, sendo mais eficiente que TreeSet e TreeMap para grandes volumes, mas menos que ArrayList, LinkedHashMap é adequado quando a prioridade é manter a ordem de inserção com acesso rápido, mas não é ideal para ordenações frequentes por atributos personalizados.

TreeMap

- Razão entre as amostras: multiplicando a entrada por 5, o tempo médio aumentou em 8,8 vezes; dobrando a entrada, a razão entre os tempos é de 2,5; e ao multiplicar novamente por 5, o tempo cresceu em 6,7 vezes.
- Analogamente à ordenação alfabética, ordenar os usuários por bonificação exige a implementação de um algoritmo de ordenação baseado em comparações, geralmente, com complexidade $O(n \log n)$.
- O TreeMap apresentou desempenho mediano e ficou em quarta posição.

TreeSet

- O TreeSet insere os armazena os seus elementos de forma ordenada, com base no parâmetro que escolhemos, em tempo de complexidade $O(\log n)$, por isso apresenta melhor desempenho em relação às demais estruturas.
- A função de desempenho é uma curva logarítmica em função do tamanho da carga armazenada.
- Neste caso em específico, o TreeSet armazena os usuários em ordem alfabética e reorganiza-os em ordem de bonificação. Por isso apresentou pior desempenho, levando 583,777 ms para execução com carga de 500000.

3.2 Comparativo de Desempenho

3.2.1 Repositórios de Dica

Para a adição de uma nova dica, o ArrayDeque apresentou o melhor desempenho (25,984 ms), seguido pela LinkedList (31,751 ms) e ArrayList (50,259 ms). Nas buscas os resultados foram bem consistentes. Para a busca de uma dica no meio da coleção, o LinkedHashMap teve o melhor tempo (0,011 ms), seguido pelo ArrayList (0,018 ms) e HashMap (0,042 ms). Na busca pela primeira dica, os melhores desempenhos foram da LinkedList (0,004 ms), LinkedHashMap (0,004 ms), HashMap (0,006 ms) e ArrayList (0,007 ms). Por fim, na busca pela última dica, o LinkedHashMap apresentou, novamente, o melhor desempenho (0,005 ms), seguido pelo ArrayList (0,008 ms), HashMap (0,008 ms) e LinkedList (0,010 ms).

Apesar de o LinkedHashMap ter um excelente desempenho nas buscas, ele teve um tempo de adição elevado (117,496 ms) em comparação com os melhores colocados nessa operação, o que compromete seu uso em cenários com muitas inserções. Já a LinkedList teve ótimo desempenho na adição e nas buscas nas extremidades, mas seu tempo para buscar no meio da estrutura (3,198 ms) é significativamente superior aos dos três primeiros colocados nesse

método. No geral, o ArrayList (estrutura original) manteve-se como a alternativa mais consistente entre todas as operações analisadas.

3.2.2 Repositórios de Usuário

Para a adição de um novo usuário os melhores resultados foram do TreeMap (210,825 ms), HashMap (231,071 ms), HashSet (280,360 ms) e LinkedHashMap (297,537 ms). No caso das buscas, o LinkedHashMap obteve o melhor tempo (0,030 ms), seguido pelo TreeMap (0,046 ms) e HashMap (0,096 ms). Então o HashMap (estrutura original) manteve-se ainda entre as opções mais rápidas para adição e busca.

Entretanto, para as operações de ordenação, os resultados variaram bastante. A ordenação por nome foi mais eficiente com o TreeSet (185,968 ms), seguido pelo ArrayList (564,937 ms) e LinkedHashMap (1.682,282 ms). Já para a ordenação por bonificação, os tempos médios foram mais equilibrados, com o ArrayList liderando (31,080 ms), seguido pelo LinkedHashMap (465,203 ms) e HashSet (532,020 ms).

Esses resultados indicam que a escolha da estrutura ideal depende da operação predominante, a mais frequente no sistema. TreeMap, HashMap e LinkedHashMap mostraram-se excelentes para cenários com foco em adição e busca. Para operações de ordenação por nome, o TreeSet e o ArrayList se destacaram. Já na ordenação por bonificação, o ArrayList e o LinkedHashMap tiveram os melhores tempos médios. Considerando todos os métodos analisados, o LinkedHashMap apresentou o desempenho mais equilibrado e consistente.

3.3 Discussão sobre os Resultados

Em geral, houve momentos em que determinadas estruturas de dados apresentaram um desempenho inesperado em relação ao que havíamos previsto inicialmente. Alguns fatores como o tamanho da carga de entrada, forma de implementação dos métodos utilizados, número de execuções das operações e características específicas do cenário de teste podem ter influenciado esse comportamento. Esses desvios ressaltam a importância de realizar testes práticos, pois o desempenho teórico esperado nem sempre reflete com precisão o comportamento real em diferentes contextos de uso.

Apesar disso, em muitos casos, os resultados obtidos confirmaram nossas expectativas, principalmente quando as diferenças entre as estruturas eram mais evidentes em termos de complexidade algorítmica. Os experimentos também evidenciaram que o desempenho de uma estrutura pode variar significativamente dependendo da operação realizada. Por exemplo, uma estrutura que apresentou ótimo desempenho em buscas pode ter tido resultado inferior em operações de inserção.

Dessa forma, os resultados obtidos foram fundamentais para identificar qual estrutura de dados é mais adequada para cada tipo de operação, como inserção, busca ou listagem. Com base no tempo médio de execução e no desempenho geral observado, é possível tomar decisões mais embasadas na escolha da estrutura mais eficiente para uma dada situação. Assim, a análise

prática influencia diretamente a escolha futura de estruturas de dados, permitindo soluções mais otimizadas e apropriadas às necessidades específicas de cada projeto.

4. Sugestão de Otimização do Sistema

Com base nos estudos analíticos realizados, concluiu-se que alterações na escolha e utilização das estruturas de dados base do código podem contribuir significativamente para a otimização do sistema, especialmente no que diz respeito à sua eficiência. A partir da análise dos métodos e das estruturas utilizadas, verificou-se que, no contexto do *Complementação*, a mudança na forma como os repositórios gerenciam seus dados pode maximizar a eficiência do sistema – considerando eficiência, neste caso, como sinônimo de maior velocidade de execução.

Assim, foi diagnosticado que a adoção de estruturas de dados distintas, utilizadas de forma conjunta e com funções específicas pré-definidas, proporciona um desempenho superior ao do modelo original. Em outras palavras, utilizar múltiplas estruturas em paralelo nos repositórios, cada uma com uma responsabilidade clara e específica, resulta em um sistema mais eficiente.

Especificamente, no caso do *UsuarioRepository*, pode-se ter a substituição do uso de `HashMap` pelo uso de `LinkedHashMap`. Essa escolha permite explorar os pontos fortes da estrutura, que segundo a análise se mantém num equilíbrio saudável. Em relação a *DicaRepository*, a estrutura utilizada originalmente (`ArrayList`) já é uma boa escolha por se manterem bons índices de execução.

Ainda sobre o segundo repositório, uma possível sugestão seria a substituição do `ArrayList` por uma combinação de `Deque` (para operações de adição) e `LinkedHashMap` (para operações de busca). Essa configuração também permite que os principais métodos operem com complexidade constante, assegurando, uma resposta igualmente rápida.

Embora a utilização conjunta de múltiplas estruturas de dados implique em um custo adicional no momento da inserção — já que o mesmo elemento precisa ser adicionado em mais de uma estrutura — essa abordagem se justifica quando o sistema realiza muitas operações de busca. Nessas situações, a leve perda de desempenho na adição é compensada pela significativa melhora na velocidade de execução das consultas.

5. Conclusão

Em suma, foram realizadas as mudanças da Estrutura de Dados (ED) presente nas classes do pacote *repository*. No projeto original foi utilizado `ArrayList` na classe *DicaRepository* e `HashMap` na classe *UsuarioRepository*. Diante disso, o mesmo projeto foi implementado utilizando diferentes EDs a fim de analisar o desempenho de cada uma realizando as mesmas operações presentes nos métodos. As EDs utilizadas foram: `ArrayList`,

Deque<ArrayDeque>, HashSet, HashMap, LinkedHashMap, LinkedList, TreeMap e TreeSet.

Foram percebidos impactos significativos quanto ao desempenho dos métodos das classes modificadas no projeto, tais impactos foram tanto positivos quanto negativos de acordo com cada estrutura de dados. Tomando como ponto de partida que as estruturas originais do projeto, como demonstrado nos gráficos de desempenho, existem ocasiões em que uma estrutura mostra-se eficiente, porém em outra ocasião a mesma mostra-se ineficiente. Um exemplo disso é o ArrayList, uma estrutura que teve um desempenho ótimo para as ordenações de usuários, porém apresentou a pior performance para a adição de novos usuários sem repetições (pelo uso do método *contains()*).

Assim, realizadas as análises de desempenho de cada Estrutura de Dados, conclui-se que não existe uma Estrutura que seja eficiente em todas as condições. Contudo, cada ED possui cenários específicos em que se mostra eficiente e outros cenários em que se apresenta ineficiente baseada na maneira como cada estrutura insere, remove, busca e lista os dados armazenados.